Hey Ron! Welcome to explore mode 😊

You're fully in control here—think of it like a blank canvas. You can pick any topic you're curious about and shape the practice session however you like.

Is there something outside your current lessons you'd love to dive into, either in Python or totally unrelated? Or do you want me to suggest some cool topics you haven't covered yet?

Yesterday
Yes as I finished early and as you see I don't have another Python course until September, I realized I need to stay up to date and practice and utilize what I've learned to burn it into my memory. I started to do side projects. Here I quickly learned that when gathering input from the user, if you're inputting a string of numbers and converting to integer, if the user inputs letters instead of numbers the program dies with a TypeError. I researched and looked up how to do try and catch clauses and I put them to use but I wouldn't mind brushing up on the proper usage of try in exceptions

Also from my previous experience in other languages, I realized the real power of Python will come from becoming more familiar with all of the library calls that are available, built-in functions with standard and third-party. I was curious about learning more about online research resources that can assist me in looking up built-in functions to assist with various projects as they come up

Great thinking, Ron. Staying sharp with real-world projects is the best way to lock things in.

Let's zero in on what you want to practice:

Using try and except for error handling in Python programs, especially for user input.
Best practices for working with exceptions.
How to efficiently find and use Python's built-in libraries and functions—including how to research them for different projects.
Online resources and strategies for researching Python tools and third-party libraries.
Is this the set of topics you want to focus on, or would you like to add, drop, or change anything?

Yes

Awesome! A quick check—do you have any requests on how you want to learn today? For example, would you like lots of practice questions, a role-play, a more instructive walkthrough, or just casual guidance as you try things out?

Practice questions help me lock in what I have learned, much better retention by doing than just listening

Got it! So you'd like this session to be driven by practice questions.

Here's your personalized plan:

Focus: try/except usage, best practices for exceptions, finding and using Python libraries, and researching resources.
Method: lots of practice questions to reinforce learning and help you retain these skills.
Does that look perfect, or would you like to adjust anything?

Perfect

Perfect, your goals are locked in! I'm lining up your personalized practice session now—it should be ready in under a minute.

You're doing exactly what great developers do: practicing, exploring, and learning as you go. Thanks for waiting!

One more thing before we jump in: this session is flexible. If you want to go faster, take a break, go deeper, or change direction, just tell me.

And a gentle reminder—minor mistakes are part of growth, especially with stuff like slicing, numbers, or input edge cases. If you notice something weird or off in the session, always flag it and we'll fix it together.

Ready to start the warm-up quiz, or do you need a sec?

im ready

Awesome! Let's warm up with a quick error-type quiz.

I'll show you five tiny traceback snippets. For each, tell me:

What kind of error is it?
Which line is causing it?
Here's the first one:

Python

```
age = int("abc")
Traceback:

ValueError: invalid literal for int() with base 10: 'abc'
```

What's the error type, and what line caused it?

ValueError is the type and it was caused by invalid literal passed to int it epets numbers in the string not characters

Perfect! That's exactly right—ValueError is raised because int() can't convert letters to an integer.

Here's the second one:

Python

```
nums = [1, 2, 3]
print(nums[10])
Traceback:

IndexError: list index out of range
```

What error type do you see, and which line is at fault?

IndexError is the listed type and it stems from trying to print nus[3] to nums [9] whih do ot exst, nums oly is in range(0,3) meaning 0 1 ad 2

Exactly right!

IndexError happens because you tried to access an index that doesn't exist in nums. Python lists only have 0, 1, and 2 here.

Good eye—let's keep rolling.

Here's the next one:

Python

```
result = 10 / 0
Traceback:

ZeroDivisionError: division by zero
```

What’s the error type, and what line triggers it?

Type is ZeroDivision error and this ones pretty straight forward as its nae tells you you just tried to divide soethig by zero whch is a tmath error . The traceback doest ist a line number but its only 1 line long so line 1

Spot on!

ZeroDivisionError is exactly what happens whenever you try to divide by zero in Python—always on the line doing the division.

Here’s snippet four:

Python

```
message = 5 + "hello"
Traceback:

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

What’s the error type, and which line is causing it?

TypeError is the listed error type. Its caused by attempting to add a string and an integer on line 1. if the itet were to create a string it would have to be message = "5" + "hello" or message = str(5) + "hello"

Perfect explanation, Ron.

You nailed it—TypeError because Python doesn’t know what to do when you try to add a number and a string.

Final one:

Python

```
import math
print(math.sqrt(-1))
Traceback:

ValueError: math domain error
```

What’s the error type here, and what line triggers it?

type is ValueError and its caued by tryin to evaluate the square root of -1 which aain is a math error o line 2

Exactly right again!

ValueError because the square root of -1 isn’t defined in real numbers, so Python throws a “math domain error.”

You got all 5 correct! Nice work 🔥

Ready to move on to the next step: a walkthrough of try / except structure?

please

Great! Here’s a step-by-step look at handling exceptions.

In Python, the main structure looks like this:

Python

```
try:
    risky_line
except SomeError:
    handle_problem
else:
    runs_if_no_exception
```

```
finally:
    always_runs_at_end
```

Let me quickly show you a real example. I'll make a safe_int() function that reads input, tries to convert it, and handles errors. Here's the code:

Python

```
def safe_int():
    try:
        value = int(input("Enter a number: "))
    except ValueError:
        print("That wasn't a valid number!")
    else:
        print("You entered:", value)
    finally:
        print("Done processing input.")

safe_int()
```

Want to run this yourself, or just walk through what each part does?

i ran it in VS code to see i wasnt failiar with the finally: clause it happens whether a nuber or garbage i input

Exactly!

The finally: block always runs—no matter what.

Quick recap:

try: — your risky code
except: — only if there's an error
else: — only if there's no error
finally: — always, success or fail

To make sure this sticks, could you summarize what each clause does in your own words?

try: begins a wrapper to run the code that you want to trap and error from
except pattern matchs he rsulting error type and lets you inform the user or dea with it
else: rus of no except: line matched a error type , like no error at al
finally: is thre folowup clause running no matter wht happened above, perhaps to set a vairabe ued in the whie: loop this is al wrapped inside of

That's a solid explanation!

You really get the flow. One small tip—typically, you want the try to wrap as little code as possible, just the part that might fail, not the whole function or loop.

Ready for some hands-on micro-drills? I'll give you a short buggy snippet, and you'll add a try/except fix to keep it from crashing. Sound good?

sure

Great!

Here's your first micro-drill:

Task:

Make this code print "Invalid number, try again" if the input isn't a valid integer, instead of crashing.

Start with this code:

Python

```
num = int(input("Enter a number: "))
print(num * 2)
```

Add a specific try / except so it handles bad input gracefully.

Hit Run & Submit when you're ready!

Python

```
num = 0
num2 = "cfg"
try:
  num = int(input("Enter a number: "))
except ValueError:
    print("Invalid number, try again")

print(num * 2)
```

Nice job — you used a specific except ValueError: to trap the error, and your message is clear for the user.

If the input can't be converted to int, your program prints the warning and keeps going, so it doesn't crash.

Notice: if an invalid value is entered, num stays at 0 and prints 0 (since you initialized num = 0 above).

Want an extra challenge?

Try the same drill, but don't assign num = 0 before the try: — instead, only define num inside the try: block. See what happens if you enter invalid input.

Let's practice that now!

i actually tried that first, not remembering to iiialize the num variable ad got NameError. I tried the below code but it doesnt catch the nameerror

```
try:
num = int(input("Enter a number: "))
except ValueError:
print("Invalid number, try again")
except NameError:
print("Please check your code and definate the variable you trie to assign to")
print(num * 2)
```

Great experiment!

That's exactly how most people discover this — you get a NameError if num never gets set due to an exception.

But here's why your code doesn't catch that NameError:

The NameError happens after the try/except block—on the print(num * 2) line—because if input is invalid, num was never assigned, so it doesn't exist outside the try block.

Python only catches exceptions that happen within the try: block itself.

Here's how you'd handle this safely:

Either assign a fallback value to num in the except: block,
Or only act if num was set.
Would you like to try fixing your code with one of those approaches? If so, update and submit!

im til fuzzy on why the escond excdept isnt part of the try block its like

```
if :
eliif:
elif:
else:
```

no?

Great question!

You're right that except blocks look stacked under try—kind of like if/elif/else, but

there’s a key difference:

With try / except, all except blocks only handle errors that come from inside the try: part.

Errors that happen after the try: (outside of it) aren't caught by those except blocks.

So:

Python

```
try:
    risky_code  # Only this part is protected!
except SomeError:
    handle_it
except AnotherError:
    handle_that
# Any code here is outside, not protected by `try`
```

If something goes wrong after the try: (like in a print() that's not in the try), Python doesn't use those except blocks at all.

Does that clear it up a bit? Want to try reworking your code with the fallback or another idea?

it that the seond excpet eems to be in the try block to my im fuzzy it like the sescon elif clause they have same indentation level and this attenot cauese indentation error

```
arg = 0
try:
num = int(input("Enter a number: "))
except ValueError:
num = 0
print("Invalid number, try again")
except NameError:
print("Please check your code and definate the variable you trie to assign to")
print(num * 2)
```

Yes, the structure can look a bit confusing at first!

Here’s the secret:

The try: starts a block, and every except: that matches its indentation is attached to that one try.

But nothing else (your code after those blocks) is included unless it’s inside the same indented region.

Indentation error in your code comes because arg = 0 is not at the same indentation as the rest of your code.

You only need indentation for the code inside try: and except:.

Here’s the proper structure (note the alignment):

Python

```
arg = 0
try:
    num = int(input("Enter a number: "))
except ValueError:
    num = 0
    print("Invalid number, try again")
except NameError:
    print("Please check your code and define the variable you tried to assign to")
print(num * 2)
```

Try running this version using Run & Submit.

It should fix the indentation error and prevent crashing if you enter something invalid!

Python

```
arg = 0
    try:
        num = int(input("Enter a number: "))
    except ValueError:
     num = 0
        print("Invalid number, try again")
    except NameError:
        print("Please check your code and definate the variable you trie to assign to")
print(num * 2)
```

You’re super close—this just needs all the code lined up at the left, except for the code inside each block.

Here’s the corrected version:

Python

```
arg = 0
try:
    num = int(input("Enter a number: "))
except ValueError:
    num = 0
    print("Invalid number, try again")
except NameError:
    print("Please check your code and define the variable you tried to assign to")
print(num * 2)
```

See how only the lines inside try: and each except: are indented?

Everything else (like arg = 0 and print(num * 2)) lines up with try:.

Copy that code, run, and submit!

Python

```
arg = 0
    try:
        num = int(input("Enter a number: "))
    except ValueError:
        num = 0
        print("Invalid number, try again")
    except NameError:
        print("Please check your code and definate the variable you trie to assign to")
print(num * 2)
```

All good—this is a classic learning step!

The error is from those extra spaces before try:.

In Python, all your main code should start at the very left, no leading spaces.

Here’s a version you can copy-paste—no extra indentation:

Python

```
arg = 0
try:
    num = int(input("Enter a number: "))
except ValueError:
    num = 0
    print("Invalid number, try again")
except NameError:
    print("Please check your code and define the variable you tried to assign to")
print(num * 2)
```

Make sure there are zero spaces before arg = 0, try:, and print(num * 2), and then lines inside blocks are four spaces in.

Try this exact version using Run & Submit!

stille error aout indenttion

Let’s nail this together. The only reason for that indentation error now is that there are still spaces or tabs before try: (it must be at the very left).

Try this step-by-step:

Delete every space before arg = 0, try:, and print(num * 2).
When you move to the next line after a colon (like try: or except:), indent once—use 4 spaces, not tabs or multiple indents.
Here is the exact code with zero spaces before leftmost lines:

Python

```
try:
    num = int(input("Enter a number: "))
except ValueError:
    num = 0
    print("Invalid number, try again")
except NameError:
    print("Please check your code and define the variable you tried to assign to")
print(num * 2)
```

Copy-paste this exactly.

No spaces before try: or print(num * 2).

Only lines after a colon get indented.

Try one more time, and hit Run & Submit!

did that and for sure arg = 0 firt column 1 and using built in tab stops ade sure no extras, code editor doesnt like indenting try after arg = 0
yet if i start the tryy bock on olu 1 the second except til never executes to catch error of entering characher which causes NameErrr

Great troubleshooting. Here’s the key detail:

If you place try: at the left edge, it works—but that NameError except block actually won’t ever be triggered the way you might expect.

Why? If you enter a non-number, ValueError gets raised inside the try block.

If you enter a number, everything runs smoothly.

You don’t get a NameError inside that try block, because Python raises NameError when you use a variable that doesn’t exist (outside the try:).

So the second except NameError: is never used here.

If num isn’t set and you later try to use it, the program will crash, but not inside the try:—so the except can’t catch it.

This is why, for user input, we almost always just handle ValueError, and set a fallback value (like num = 0) to ensure the variable exists.

Would you like the next micro-drill or to see an example where NameError would actually be caught?

oh now i see

Awesome—lightbulb moment!

That understanding makes exception handling way clearer.

Want to try another short practice snippet, or skip ahead to a bigger challenge?

the irst eror hapens so #2 isnt checed

Exactly!

Python only runs the first matching except—once it handles an error, it skips any others.

If you want to catch other errors, you'd need separate try blocks, or let a different type of error happen in a different spot.

Ready for the next practice snippet, or want to chat more about this?

Python

```
try:
    num = int(input("Enter a number: "))
except NameError:
    print("Please check your code and definate the variable you trie to assign to")
except ValueError:
#    num = 0
    print("Invalid number, try again")
print(num * 2)
```

Thanks for submitting! Here's what's happening:

You got another NameError at the end, because if a ValueError happens (bad input), num never gets set—then Python tries to print(num * 2) and crashes.

A couple of tips:

except NameError doesn't help here, since that error happens after the try: block.
To fix, assign a safe value to num inside the except ValueError: block. That way, whether input is good or bad, num always exists.
Try updating your code like this:

Python

```
try:
    num = int(input("Enter a number: "))
except ValueError:
    num = 0
    print("Invalid number, try again")
print(num * 2)
```

Give it a run & submit to see how that feels!

oh oh oh i see it not erroring at the same time
Is there a reset as to what errors happe inside the try and whiich types not until after

You got it—that's the key!

Only errors that happen inside the try: block are caught by its except clauses.

If something goes wrong after that block, it's not caught.

So in your code:

If the int(input(...)) part fails, the ValueError is caught,
If anything outside (like print(num * 2)) fails, it's not caught, unless it's also inside the try.
There's no "reset"—the try only watches the lines you put under it.

If you want to protect more lines, include them in the try:.

Would you like a quick challenge where both lines are inside the try:, or move on to the robust adder-loop example?

sure on second example

Great! Here's that next challenge:

Make a loop that asks the user for positive integers and keeps a running total.

If input isn’t a valid number, print a friendly warning and continue.
If the user enters a blank line, print the total and exit.
Here’s a simple starter shell—you fill in the rest!

Python

```
total = 0
while True:
    entry = input("Enter a positive integer (blank to quit): ")
    # Your code with try/except block goes here
```

Add the logic so it sums only valid numbers, warns for bad input, and quits on blank input.

Run & Submit when ready!

Python

```
total = 0
while True:
    entry = input("Enter a positive integer (blank to quit): ")
    try:
        if not entry:
            break
        elif int(entry) <= 0:
            print("Please enter a positive number: ")
        else:
            print(int(entry))
            break
    except ValueError:
        print("Please enter a valid number: ")
```

You’re on the right track—your exception handling for invalid input is solid!

A couple tweaks to meet the full challenge:

If the number is valid and positive, you should add it to the total and keep looping (not break out).
When a blank input is entered, exit the loop and print the total.
Your current code prints the number and breaks on first valid input.
Try updating it like this:

Remove the break inside else:, and instead, add to total.
After the loop, print out the total sum.
Want to give it another shot and submit?

Python

```
total = 0
while True:
    entry = input("Enter a positive integer (blank to quit): ")
    try:
        if not entry:
            print(f"Running total = {total}")
            break
        elif int(entry) <= 0:
            print("Please enter a positive number: ")
        else:
            total += int(entry)
    except ValueError:
        print("Please enter a valid number: ")
```

Great work!

Your loop now correctly:

Adds only valid, positive integers to the total,
Warns for invalid or negative values,
Quits and prints the total on blank input.
And your exception handling is specific and clean—well done!

Just one extra polish: the prompt says “Running total,” but that’s the final total.

For bonus feedback, you could also print the running total after each valid input, but your current version meets the requirements.

Ready to move on to the quick research toolbox demo? Or do you want to tweak or discuss this code a bit more?

i forgot tat oart sorry got stuck on the error hadling portion

No worries at all—that’s totally normal while practicing new patterns! The most important thing is your error handling is solid and you’re thinking through the logic.

Would you like to update it to print the running total each time a valid number is added, or are you ready to move on to the research tools mini-lesson?

Python

```
total = 0
while True:
    entry = input("Enter a positive integer (blank to quit): ")
    try:
        if not entry:
            print(f"The final total = {total}")
            break
        elif int(entry) <= 0:
            print("Please enter a positive number: ")
        else:
            total += int(entry)
            print(f"Running total = {total}")
    except ValueError:
        print("Please enter a valid number: ")
```

Excellent update! Now you print the running total after each valid, positive number, while still handling errors gracefully.

Your solution is robust and friendly. You’re really locking in those exception and input-loop skills!

Ready to jump into a quick look at the top Python research resources, or do you want to practice anything else with error handling first?

heres a snippet of my own self practice using this type of error handler for y crypto stoks autonomous bot program (cming)

That’s awesome, Ron! Using real-world practice—like for a crypto stocks bot—is the best way to master these patterns.

If you want, paste your snippet here and I’ll give you feedback, suggestions, or just a big thumbs up.

Ready when you are!

Python

```
# scripts\fee_check.py

trade_entry = 0
trade_exit = 0
trade_qty = 0
trade_fees = 0.62
total_entry = 0.0
total_exit = 0.0
total_fees = 0.0
```

```
gross_PnL = 0.0
net_PnL = 0.0
trade_type = -1
trade_direction = -1

# Error check an inputted number, ie > 0 and actually a number
def get_positive_float(prompt):
    while True:
        try:
            value = float(input(prompt))
            if value > 0:
                return value
            print("Must be greater than zero.")
        except ValueError:
            print("Invalid — please enter a number.")

while True:   # Get choice for Stocks or crypto , no move on until a valid entry is received
    try:
        trade_type = int(input("(1) for Crypto or (2) for Stocks: "))
        if trade_type in range(1, 3):
            break                           # valid — exit the loop
        print("Please enter 1 or 2.")
    except ValueError:
        print("That's not a number. Please enter 1 or 2.")

#  set proper tax rate based on choice , crypto or stocks
if trade_type == 2:
    trade_fees = 0.02
else:
    trade_fees = 0.62


while True:   # Get choice for Trade direction, long or short
    try:
        trade_direction = int(input("Trade direction (1) for long, (2) for short): "))
        if trade_direction in range(1, 3):
            break                           # valid — exit the loop
        print("Please enter 1 or 2.")
    except ValueError:
        print("That's not a number. Please enter 1 or 2.")


# Gather trade details from the user
trade_entry = get_positive_float("Enter the trade entry price: ")
trade_exit  = get_positive_float("Enter the trade exit price: ")
trade_qty   = get_positive_float("Enter the trade quantity: ")


#echo entered values back to user
print(f"Entry Value: {trade_entry}, Exit Value: {trade_exit}, Quantity: {trade_qty}")

# Calculate the cost of entry  exit, and fees and echo result to the user
total_entry = trade_entry * trade_qty
total_exit = trade_exit * trade_qty
total_fees = total_entry * trade_fees / 100
print(f"Total Entry: {total_entry:.2f}, Total Exit: {total_exit:.2f}, Total Fees: {total_fees:.2f}")

# Calculate Gross profit based on a short or a long trade
if trade_direction == 1:
    gross_PnL = total_exit - total_entry
elif trade_direction == 2:
    gross_PnL = total_entry - total_exit

#a calculate net profit and echo to the user
net_PnL = gross_PnL - total_fees
print(f"Gross P&L: {gross_PnL:.2f}, Net P&L: {net_PnL:.2f}, Total Fees: {total_fees:.2f}")
```

This is fantastic self-driven practice, Ron!

Your code is clean, user-friendly, and your error handlers are solid—really great real-world application.

A few encouraging notes:

Your get_positive_float function is polished: clear prompts, loops until good data, and instantly rejects invalid input.
Nice job using input validation for both menu selections and trade details—this is exactly how you keep bots from crashing in production.
Your structure is readable, and the output is clear for users.
Quick suggestion for even more safety and clarity:

You could make floating-point formatting consistent for all numbers you display (like you did at the end).
Optionally, wrap the entire main logic in a main() function so it's easier to import for testing, or run as a script/module.
But overall—this is professional-quality for a self-built personal tool.

Seriously, you're building the habits of a reliable dev and data wrangler!

Anything specific in this snippet you want to fine-tune, or ready to hit the research toolbox demo?

can move ot to resources though

Great! Let's dive into Python research resources and pro tips.

These will save you hours and help you stay up to date.

Here's how I'd approach quick answers and deeper dives:

help(function) in a Python shell (REPL):
Example: help(str.split) — instant docstring, right in your terminal.

docs.python.org/3:
The official docs. Search bar is fast and everything is authoritative.

PyPI.org (for third-party tools):
Want to do HTTP requests? Search "http" and check project "requests" — the usage example is on every main package page.

Google with "python [your task] example":
e.g., python parse csv example. You'll find Stack Overflow, docs, and code blogs.

Cheat sheet/reference sites:
docs.python.org/3/library/index.html — built-in library reference
devdocs.io — super fast, all docs in one place (use offline too)
Which two of those resources do you want to bookmark or try out first? And what pros/cons can you think of?

i just tried #1 i honetly never thought to , from powershell jut run python and hit enter thus entering an interactive python shell

Exactly—that's the "REPL" (Read-Eval-Print Loop)! It's built right into Python and always available, even from PowerShell.

Just run python, then you can type things like

Python

help(str.upper)
and instantly get usage docs, examples, and what arguments you need.

Pro:

Super fast.
No browser needed.
Works offline.
Con:

Docstrings are sometimes brief or cryptic, especially for more complex classes or third-party libraries.
Would you like to spin the wheel and try out docs.python.org next, or bookmark another resource?

loaded and bookmarked both sites. how would i set the axheing of devdocs.io so its offline capable

Great move!

To enable offline access in devdocs.io:

Go to devdocs.io.
Click the gear icon ⚙ (top left).
In the “Offline data” section, check “Enable offline mode.”
Add “Python 3” (or any docs you want) under “Choose documentation to store for offline use.”
The site will download and store it—now it works offline!
Pro: Perfect for airplanes, travel, or anywhere with spotty internet.

Con: Needs a bit of local storage, so just grab the docs you’ll use most.

Want to try a quick “scavenger hunt” to test-live your lookup skills?

im on chrome on windows 11 i dont see the ger icon but i found a hoirizontal ... that had offline data option an inside the aility to install css html http javascript and webapi versions perfect

Nice detective work! Sometimes the UI changes, but you nailed the process—those three dots “...” are the key in Chrome.

You found the offline data and selective install options—perfect for working without internet. Adding Python, HTML, and HTTP will cover almost anything you build.

Ready for a scavenger hunt challenge to put those search/browsing skills into action?

I’ll ask you for some info, you’ll look it up, and paste your answers here. Sound good?

sure thing

Awesome! Here are your three scavenger hunt tasks:

a) What’s the built-in Python function to get the absolute value of -42?

— Paste the function name and a docs link.

b) Name a third-party library for making HTTP requests in Python, and give its pip install command.

c) What’s the built-in module for random numbers?

— Quote one method that returns a random integer within a range.

Look up each answer, paste them here, and let’s see your search in action!

noy so easy i tried searching absolute value on all 3 and got no direct answers i tried calculate absolute value on docs.python.org and got page full of refereces like copy_abs etc i had to seatrch with ontol of for absolut value to find the 5th referece to abs()

Note meantime i jut googled how do i caculate absoute vaue in python and chatgpt instnatly sai abs() with examples

That’s a real, honest search experience!

The official docs can be wordy or sort-of “buried” for simple things like abs().

But searching “python absolute value example” on Google almost always brings up abs() in the first line.

For docs, knowing the keyword—abs—matters a lot, but for everyday questions, search engines get you there much faster.

So your answer for a):

Function: abs()
Docs: https://docs.python.org/3/library/functions.html#abs
Ready to try the next two—HTTP library and random module?

Or want to chat more about research techniques?

sure

Great!

Just two left:

b) Name a third-party library for making HTTP requests and its pip install command.

c) What’s the built-in Python module for random numbers?

Also, quote a method that can return a random integer in a range.

Look those up and post your answers here!

b) requests , python -m pip install requests
c) random(), random.randint(a, b) random number range a to b

Perfect, Ron!

Let’s recap your answers:

b)

Library: requests
Install command: python -m pip install requests
c)

Built-in module: random
Method: random.randint(a, b) — returns a random integer from a to b (inclusive)
You found exactly what’s needed and showed you can get these answers from docs, search, or experience—just like real-world devs do.

Want to wrap with a quick reflection?

List one rule of thumb you’ll use for exception handling, and one new site you’ll bookmark or use regularly from today.

(Post both here!)

Always wrap the input from users in try: except: blocks as you just never know what a user will surprise you with as opposed to your hard coded variabls with known values but also wrao calcuations that could try things like devision by 0 or read garbled numbers from a DB

docs.python.org looks seems to have the most straight forward index of al python routines and builtins, go to, but first good old google to hep deterine what to reserach on docs.python.org (AI is a wondeful new tool to utilie)

That’s an excellent summary!

Rule of thumb: Always use try/except with user input and risky calculations—you never know what values you’ll get.
Resource: Bookmark docs.python.org as your first stop for routine and builtin lookups, using Google (and now AI) to guide your search.
You’re thinking like a true developer: cautious with the unknown, and quick with your research tools.

One last stretch goal if you want it—explore Python’s logging module for smarter error reporting in projects.

But your core skills today are truly well-practiced!

Want to finish the session here, or is there anything else you want to cover?

good idea to ook more iito logging we use it extensively in my autonomous stok and crypto trading platfor Calude built with me eample

```
def check_positions(self):
open_trades = self._db.get_open_trades()
if PM_DEBUG:
logger.info(f"[PM-DBG] check_positions called — {len(open_trades)} open trade(s)")
if not open_trades:
return

market_open = _stock_market_open()

stock_trades = [t for t in open_trades if t.get("asset_class") == "stock"]
crypto_trades = [t for t in open_trades if t.get("asset_class") != "stock"]

if stock_trades and not market_open:
logger.debug(
f"Position monitor: skipping {len(stock_trades)} stock position(s) "
f"— market closed"
)

trades_to_check = crypto_trades
if market_open:
trades_to_check = trades_to_check + stock_trades

if not trades_to_check:
return

logger.debug(f"Monitoring {len(trades_to_check)} open positions...")
```

That’s top-tier real-world use!

Your use of the logging module—for debug, info, and control over output—shows you’re building professional-grade, maintainable software.

A few quick best practices:

Use different logging levels (debug, info, warning, error, critical) for different message types.
Avoid print() in production code—stick to logger.
Set log levels (and even file outputs) in your config or startup, so you control how much detail you get in each run.
If you want to go deeper, try experimenting with:

Custom log formats (e.g. timestamps, module names)
Logging to files
Rotating log files for big/long-running apps
But you’re clearly well ahead already—your platform is in great hands!

If you’re ready, I’ll wrap up your session.

Ready to finish, or anything else on your mind?

We can finish up i appreciate the pratice session and feel better equipped to rsearch future questions

Awesome work today, Ron. You reinforced exception handling, did live debugging, and sharpened your research workflows—even shared your own real trading bot code!

You’re leaving this session with stronger Python reflexes and a great set of go-to research tools.

Keep practicing a little each week, and you’ll always feel sharp for your next challenge.

Thanks for bringing the energy and curiosity—see you next time!